Employee Attendance & Analytics API

Python 3.11+, FastAPI | MongoDB / MongoDB Atlas via asynchronous PyMongo | OpenAPI and Swagger UI

Entry point: app/main.py | Report generated: 15 October 2026

Contents

1. Executive Summary 1
2. Project Objectives and Features 1
3. Technology Stack 1
4. System Architecture and Data Flow 2
5. MongoDB Collections, Schema and Indexes 2
6. Complete API Documentation 2
7. Attendance Business Rules and Calculations 3
8. MongoDB Aggregation and Analytics 4
9. Input Validation and Error Handling 4
10. Concurrency and Data Integrity 4
11. API Testing and Verification Results 4
12. Environment Configuration and Setup 5
13. Design Decisions 5
14. Conclusion 5

1. Executive Summary

The service manages employees, daily attendance (punch-in and punch-out), manual attendance corrections with an audit trail, and monthly and daily analytics. It is a FastAPI application backed by MongoDB through PyMongo's native asynchronous client (`AsyncMongoClient`) and starts with `uvicorn app.main:app --port 8000`. All twelve contract endpoints are implemented in `app/main.py`. On the recorded evidence the API works end to end: the author manually checked health, employee creation, punch-in, punch-out, filtering, regularization, monthly analytics and the department summary, and the README records 47 passing automated tests (41 mock, 6 live MongoDB). This shows the endpoints run and the covered scenarios behave as intended; it is not exhaustive testing of every input.

Source basis. Prepared from `app/main.py`, `requirements.txt`, `README.md`, `REVIEW.md` and `DECISIONS.md`. `openapi.yaml`, `DATA_MODEL.md` and `tests/` were not available, so schemas come from the code and README and test results are quoted as the README records them. Cross-check against `openapi.yaml` and re-run the suite before submission.

2. Project Objectives and Features

- Employees identified by `emp_code` (MongoDB `_id` is never exposed); paginated listing.
- Punch-in/out with IST attendance dates, overnight shifts, late, overtime, work-hours and half-day calculation; filtered attendance listing.
- Regularization with a history of every change; four analytics pipelines in MongoDB; an admin explain endpoint; safe automatic index creation at startup.

3. Technology Stack

Component	Detail
Language / framework	Python 3.11+ (suite recorded on 3.14.5); FastAPI >= 0.110; Uvicorn >= 0.29; Pydantic >= 2.6
Database / driver	MongoDB 6.0+ (Atlas via `mongodb+srv://`); `pymongo[srv]` >= 4.13 using `AsyncMongoClient` (Motor is not used)
Other	`python-dotenv` >= 1.0 (environment overrides `.env`); `httpx` >= 0.27 listed in `requirements.txt`; Swagger UI at `/docs` from the generated OpenAPI schema

4. System Architecture and Data Flow

```
Client -> FastAPI route -> Validation (Pydantic / Query; 422) -> Business rules (IST, late, overtime, hours)
  -> MongoDB operation (insert_one / update_one / find / aggregate) -> Serialization (no _id, epoch ms)
  -> JSON response
```

- **Lifespan:** requires `MONGO_URI` and `MONGO_DB`, creates `AsyncMongoClient(tz_aware=True, serverSelectionTimeoutMS=8000)`, pings, creates indexes; any failure stops startup. `.env` is loaded with `override=False`; no default URI exists in the code.
- **Time:** API uses epoch milliseconds; MongoDB stores UTC BSON datetimes; attendance dates and shifts use fixed IST (UTC+05:30). Derived fields are computed on write and stored.

5. MongoDB Collections, Schema and Indexes

Collection	Fields
`employees`	`emp_code` (unique; `EMP` + 4-6 digits), `name`, `email`, `department`, `shift_start`/`shift_end` (`HH:MM`, default 09:30/18:30, must differ), `joined_on` (`YYYY-MM-DD`), `created_at` (UTC datetime)
`attendance_logs`	`emp_code`, `date` (unique together), `status`, `punch_in`/`punch_out` (UTC datetime or null, whole seconds), `work_hours`, `late_minutes`, `overtime_minutes`, `half_day`, `history[{at, by, reason, changes}]`

Nine named indexes are created at startup only if missing (versioned list `INDEX_MIGRATIONS`). A same-name index with a different definition stops startup with an error; nothing is dropped or replaced.

Collection	Index	Keys	Purpose
`employees`	`employee_code_unique`	`emp_code` (unique)	Identity and lookup
`employees`	`department_employee_code`	`department`, `emp_code`	Department listing order
`employees`	`department_joined`	`department`, `joined_on`	Department headcount
`employees`	`employee_joined`	`joined_on`	Eligibility by join date
`attendance_logs`	`employee_date_unique`	`emp_code`, `date` (unique)	One record per employee per date
`attendance_logs`	`employee_punch_in`	`emp_code`, `punch_in` desc	Latest punch-in for punch-out
`attendance_logs`	`attendance_date_employee`	`date` desc, `emp_code`	Attendance listing order
`attendance_logs`	`attendance_date_late`	`date`, `late_minutes`	Date-range late analytics
`attendance_logs`	`attendance_status_date`	`status`, `date` desc, `emp_code`	Status-filtered queries

6. Complete API Documentation

Read from `app/main.py` and the README. Errors use `{"detail": ...}`; validation failures return 422; timestamps are integer epoch **milliseconds**. Paged endpoints use `page` (>= 1, default 1) and `page_size` (1-100, default 20) and return `{items, total, page, page_size}`.

Endpoint	Request	Success	Errors and notes
`GET /health`	None	200 `{"status":"ok"}`	503 `MongoDB is unavailable` if no client or ping fails
`POST /employees`	Body: `emp_code`, `name`, `email`, `department`, `joined_on` (shifts optional)	201, employee with `created_at` (ms)	409 duplicate `emp_code` (unique index); 422 bad code, email, date, shifts
`GET /employees`	Query: `department`, paging	200, sorted by `emp_code`; `total` after filter	422 bad paging
`POST /attendance/punch-in`	Body: `emp_code`; optional `punched_at` (ms), `status` (`PRESENT` default, `WFH`, `ON_DUTY`)	201, attendance record	404 unknown employee; 409 already punched in; 422 non-integer, seconds-style or out-of-range timestamp, bad status
`GET /attendance`	Query: `emp_code`, `date_from`, `date_to`, `status`, paging	200, date desc then `emp_code`	422 bad date, reversed range, bad status or paging
`POST /attendance/punch-out`	Body: `emp_code`; optional `punched_at`	200, updated record	404 no employee / no punch-in; 409 already punched out; 422 order or over 24 h
`PATCH /attendance/{emp_code}/{date}`	Body: optional `status`, `punch_in`, `punch_out`; required `reason` (5-200), `regularized_by`	200, record with new history entry	404 missing employee/record; 422 invalid combination or no-op; 409 concurrent change

Endpoint	Request	Success	Errors and notes
`GET /analytics/employees/{emp_code}/monthly`	Query: `month` (`YYYY-MM`)	200: `working_days`, `present_days`, `leave_days`, `late_count`, `total_late_minutes`, `total_overtime_minutes`, `attendance_pct`	404 unknown employee; 422 bad month
`GET /analytics/departments/summary`	Query: `month`; optional `department`	200 `{month, items}`: `headcount`, `present_days`, `avg_work_hours`, late and leave counts	422 bad month
`GET /analytics/leaderboard/late`	Query: `month`; `limit` (1-50, default 10); optional `department`	200 `{month, items}`: `rank`, `emp_code`, `name`, `department`, `total_late_minutes`, `late_count`	422 bad month or limit
`GET /analytics/departments/{department}/trend`	Query: `from`, `to` (max 92 days)	200 `{department, items}`: `date`, `is_working_day`, `headcount`, `present_count`, `late_count`, `attendance_rate`, `moving_avg_7d`	404 unknown department; 422 bad or reversed range
`GET /admin/explain/{endpoint}`	`endpoint` in `attendance_list`, `employee_monthly`, `department_summary`, `late_leaderboard`, `department_trend`; that query's parameters	200 `{endpoint, collection, explain}` (`executionStats`)	422 unknown endpoint or missing parameters

```
POST /attendance/punch-in   {"emp_code": "EMP0001", "punched_at": 1783312500000, "status": "PRESENT"}
POST /attendance/punch-out  {"emp_code": "EMP0001", "punched_at": 1783344600000}
PATCH /attendance/EMP0001/2026-07-06
   {"status": "WFH", "reason": "Corrected attendance status", "regularized_by": "hr.admin"}
```

7. Attendance Business Rules and Calculations

Rule	Implementation
Time (R1)	Epoch ms in the API, UTC BSON in MongoDB, fixed IST for dates and shifts. Instants truncated to whole seconds before calculation; the truncated value is stored and returned. Seconds-style, float and string timestamps are rejected.
Overnight (R1)	Overnight when `shift_end <= shift_start`; a punch-in earlier than `shift_end` belongs to the previous day's shift.
Late (R2)	With s seconds after shift start: if `s > 600`, `late_minutes = floor(s / 60)` (total minutes, not minutes past grace); else 0.
Overtime (R3)	Whole minutes after shift end (next day for overnight); recorded only if >= 30, else 0.
Hours / half day (R4, R5)	Elapsed seconds / 3600 with `Decimal`, half-up, 2 decimals; null until punch-out; limit 24 h. `half_day` when rounded hours < 4.50 (exactly 4.50 is not); counts 0.5 present day.
Presence, working days (R6, R7)	`PRESENT`, `WFH`, `ON_DUTY` present; `ABSENT`, `LEAVE` not; a missing record is not `ABSENT`. Mon-Fri, no holidays, counted from the later of month start and `joined_on`; weekend records still add to late and overtime totals.
Numbers, headcount, paging (R8-R10)	Reported values 2 decimals half-up, rates 4. Headcount counts employees with `joined_on` on or before period end, including those with no logs (per day in the trend). Paging from 1, default 20, max 100, `total` after filters.

Example	Result
22:00-06:00 shift, punch-in 05:30 IST on 8 July	Attendance date 7 July
09:30 shift: punch-in 09:40:00 / 09:40:01 / 10:15:59	0 / 10 / 45 late minutes
09:30-18:30 shift: punch-out 18:59:59 / 19:00:00	0 / 30 overtime minutes
54 s of work; 16182 s (4.495 h); 16164 s (4.49 h)	0.02 h (half-up); 4.50 h, not half day; half day
18 working days, 1.5 present days	`attendance_pct` 8.3333

8. MongoDB Aggregation and Analytics

Endpoint	Pipeline (stages used in `main.py`)
Monthly	`$match`, `$set` weekday (`$dayOfWeek`), `$group` totals; working days and percentage computed from the join date.
Department summary	Starts from `employees` (`$match` joined by period end), `$lookup` with per-employee sub-pipeline, `$ifNull` for no-log employees, `$group` by department, `$sort`, `$project`. Average = hours sum / hours count over records, not an average of employee averages.
Leaderboard	`$match` late > 0, `$group`, `$lookup`, `$unwind`, `$setWindowFields` with `$rank` on total late minutes, `$match` rank <= limit, `$sort` (total desc, `emp_code`), `$project`.
Trend	`$match`, `$lookup`/`$unwind`, `$group` by day, `$unionWith` generated days (`$documents`, `$range`, `$dateAdd`), `$densify`, headcount `$lookup`, decimal half-up rate, `$setWindowFields` `$avg` over `documents: [-6, 0]`.

Ranking: `$rank` gives 1, 2, 2, 4; array position would split ties. The limit is a rank cutoff applied after ranking, so ties at the cutoff are all returned and only employees with a late record appear. **Trend:** one row per calendar day including weekends and empty days; rate only on working days with headcount; the moving average uses non-null rates, so the window is shorter at the start. **Explain:** the README records five explain patterns with `IXSCAN` and no `COLLSCAN`, and `employee_date_unique` used by the summary lookup (9 keys, 9 documents examined, 7 returned). This is for the tested data, not a benchmark.

9. Input Validation and Error Handling

200/201 success; **404** missing resource; **409** duplicate employee, second punch-in or punch-out, lost concurrent regularization; **422** invalid input (formats, timestamps as `StrictInt` within 1e11 to 4.1024448e12 ms, status, paging, ranges, no-op regularization); **503** `/health` with MongoDB down. `_id` is stripped from responses and duplicate-key errors become 409. **Limitation (by inspection, untested):** only `/health` maps general MongoDB errors to 503; a database outage during other requests would surface as a server error.

10. Concurrency and Data Integrity

Operation	Mechanism	Recorded result (README)
Employee create	Unique index on `emp_code`	8 requests: one 201, seven 409
Punch-in, same employee and date	Unique `(emp_code, date)` index; duplicate key becomes 409	8 requests: one 201, seven 409; one record
Punch-out	`update_one` filtered on open record; `modified_count` checked	8 requests: one 200, seven 409
Regularization	Update filter holds loaded values and history; `$push` entry	2 requests: one 200, one 409; one history entry

No transactions or distributed locks are used; guarantees rest on unique indexes and atomic single-document conditional updates.

11. API Testing and Verification Results

Suite (README): `python -m unittest tests.test_main -v` (mocked), `python -m unittest discover -s tests -p test_mongo_concurrency.py -v` and `python test_concurrent.py` (live MongoDB), `python -m unittest discover -s tests -v` (all). Mock tests are fast and simulate failures but cannot prove real queries or indexes; live tests use `MONGO_TEST_DB` (default `<MONGO_DB>_test`) and clean up their own documents. **Recorded:** 47 passed (41 mock, 6 live) and `test_concurrent.py` passed; a fresh environment installed `requirements.txt`, a source-only copy started and returned 200 from `/health`, and startup failed with `ServerSelectionTimeoutError` when MongoDB was unavailable. These are recorded results, not re-run for this report.

Endpoint	Evidence	Status
`GET /health`	Manual check, both states; unit test for 503	Verified
`POST /employees`	Manual check incl. invalid code; live concurrency	Verified
`GET /employees`	Mocked HTTP tests (README); no manual check listed	Covered by tests
`POST /attendance/punch-in`, `punch-out`	Manual checks (late, 409, hours, half day); live concurrency	Verified
`GET /attendance`	Manual check of filtering and pagination	Verified
`PATCH /attendance/...`	Manual single regularization and history; live concurrency	Verified
`GET /analytics/employees/.../monthly`	Manual check	Verified
`GET /analytics/departments/summary`	Manual check, code-level Copilot audit, live explain	Verified
`GET /analytics/leaderboard/late`, `.../trend`	Queries run under recorded live explain tests; no manual response check listed	Executed; edge cases not individually confirmed
`GET /admin/explain/{endpoint}`	Recorded live explain tests (five patterns)	Verified

Completeness. All twelve endpoints are implemented and, on the recorded evidence, execute successfully, so the suite works end to end. "Verified" means a recorded or manual check of the behaviour listed, not exhaustive testing. Not done: a 100,000-record benchmark, confirmed leaderboard and trend edge cases, database-error tests on routes other than `/health`, and an `openapi.yaml` cross-check for this report.

12. Environment Configuration and Setup

```
python -m venv .venv                        # macOS/Linux: python3 -m venv .venv
.\.venv\Scripts\Activate.ps1                # macOS/Linux: source .venv/bin/activate
python -m pip install -r requirements.txt

# .env (never committed) or real environment variables, which take precedence:
MONGO_URI="mongodb+srv://<username>:<password>@<cluster-address>/?retryWrites=true&w=majority"
MONGO_DB="<database-name>"

uvicorn app.main:app --port 8000          # then open http://127.0.0.1:8000/docs and GET /health
```

In Atlas: create a cluster and a database user with read/write and index rights, allow only your IP under Network Access (avoid `0.0.0.0/0`), copy the Python connection string, and URL-encode special characters in the password (for example with `urllib.parse.quote_plus`).

13. Design Decisions

Decision	Reason
Nine named, non-destructive indexes	Support the queries and enforce uniqueness; conflicts are reported, never dropped.
Unique index and conditional updates as race guards	Application checks alone can both pass; the database lets only one write win.
`$rank` and a rank-cutoff limit	Ties share a rank and ties at the cutoff are all returned; `emp_code` keeps display order stable.
Summary starts from employees; async client; fail fast on configuration	No-log employees stay in headcount; calls do not block the event loop; a wrong state is never served.

`DECISIONS.md` also proposes scaling steps (replicas, queue with retries, rate limiting); these are not implemented. `REVIEW.md` records 20 historical starter findings that cannot be re-verified because the original starter is gone.

14. Conclusion

The project delivers the twelve contract endpoints with the required business rules, aggregation-based analytics and database-level concurrency protection. Based on the author's manual checks and the recorded results, the API suite runs end to end; the evidence covers the tested scenarios, not every input. Before submission: re-run the suite and record one test count, cross-check schemas against `openapi.yaml`, run `/health` and the explain endpoint once against Atlas, and confirm `.env` is excluded from the repository.